JUG Istanbul Tech Day 2026

Paylaşımlı Veritabanından Kurtulmak

70 uygulamada servis ve buffer cache ile referans veri dağıtımı

İbrahim Atay, Senior Software Architect

linkedin.com/in/ibrahimatay · github.com/ibrahimatay · ibrahimatay.com



Gündem

01 **Problem Tanımı**
En basit verinin 70 uygulamaya dağıtımı

02 **Eski Model ve Bedeli**
Paylaşımlı veritabanı ve gizli maliyetleri

03 **Yeni Model**
Servis + buffer cache ve sorgu yolu

04 **Tasarım Kararları**
Servis, cache, REST ve MQ tartışmaları

05 **Göç ve Dersler**
Neyi farklı yapardık, taşınabilir desen



Referans Veri
Servis + Buffer Cache

Kur bilgisi bir bankadaki en basit veri gibi görünür

2

günde güncelleme

birkaç

sayıdan oluşan veri

70+

ona ihtiyaç duyan uygulama

Bu birkaç sayıyı nasıl dağıttığınız, bir **mimari probleme** dönüşür.

Aynı Veri, Üç Tüketim Biçimi

Form Ekranları

Tek tarih, tek kur. Kullanıcı girdisi alma ve doğrulama.

Gridler

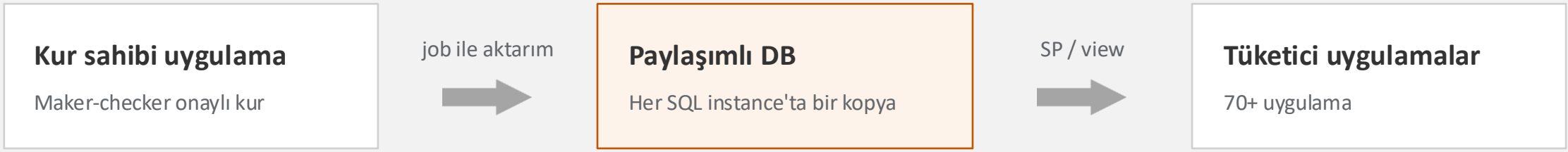
Kurla hesaplanan satırlar; her satır farklı bir değer tarihinin kurunu ister.

Batch Job'lar

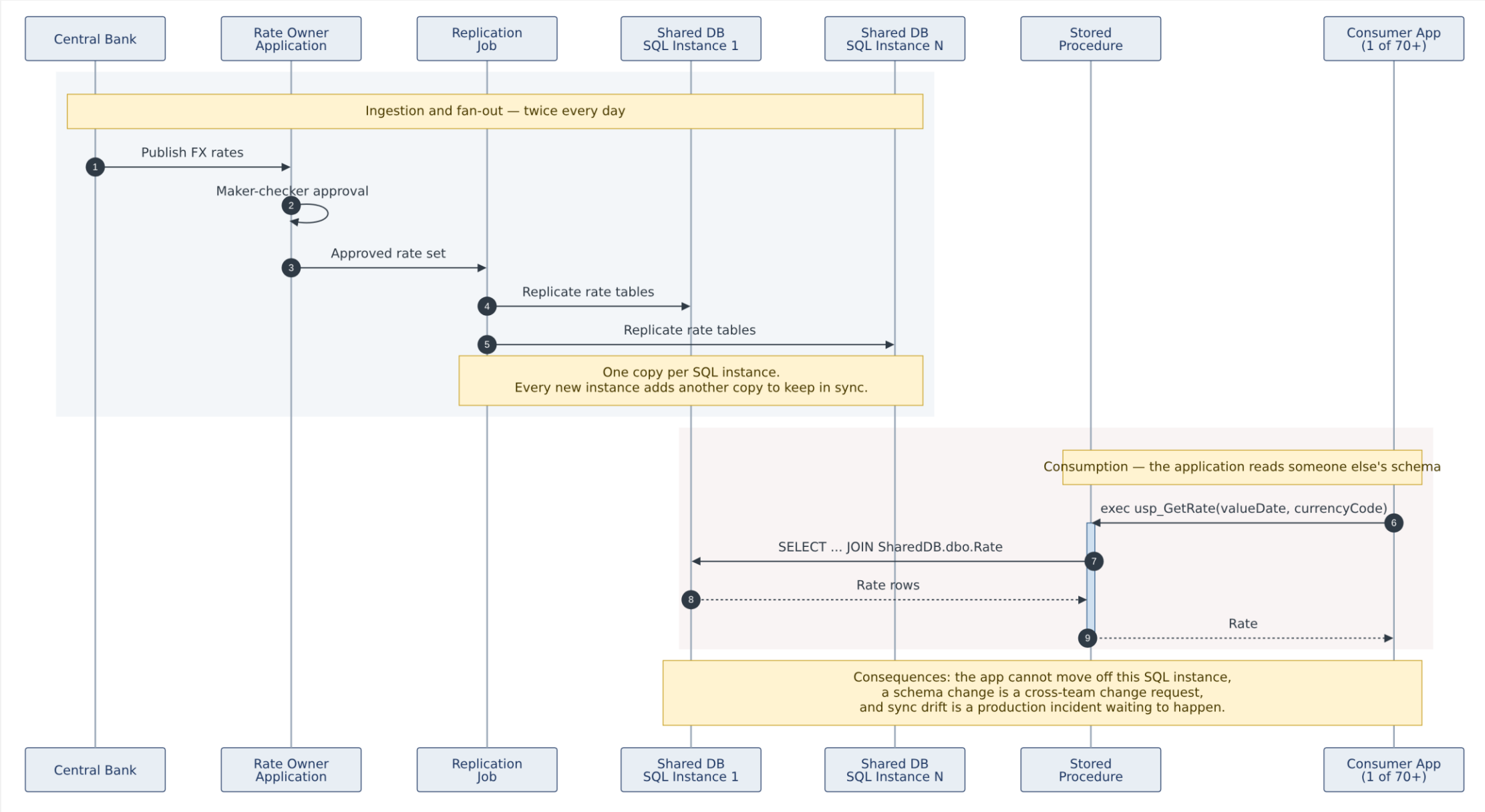
Gece çalışan toplu işler ve veri kontrolleri.

⚠ Portföydeki uygulamalarda kur kullanan ekran sayısı tam olarak bilinmiyor.

Eski Model: Kopyala ve SP ile Oku



Çalıştı. Yıllarca.


Central Bank
Rate Owner Application
Replication Job
Shared DB SQL Instance 1
Shared DB SQL Instance N
Stored Procedure
Consumer App (1 of 70+)
Ingestion and fan-out — twice every day
1 Publish FX rates
2 Maker-checker approval
3 Approved rate set
4 Replicate rate tables
5 Replicate rate tables
One copy per SQL instance.
Every new instance adds another copy to keep in sync.
Consumption — the application reads someone else's schema
6 exec usp_GetRate(valueDate, currencyCode)
7 SELECT ... JOIN SharedDB.dbo.Rate
8 Rate rows
9 Rate
Consequences: the app cannot move off this SQL instance,
a schema change is a cross-team change request,
and sync drift is a production incident waiting to happen.
Central Bank
Rate Owner Application
Replication Job
Shared DB SQL Instance 1
Shared DB SQL Instance N
Stored Procedure
Consumer App (1 of 70+)

Gizli Bedel: Sessizce Tıkanan Yol

Her yeni SQL instance

Senkron tutulacak bir kopya daha → bakım maliyeti sürekli büyüyor

Her şema değişikliği

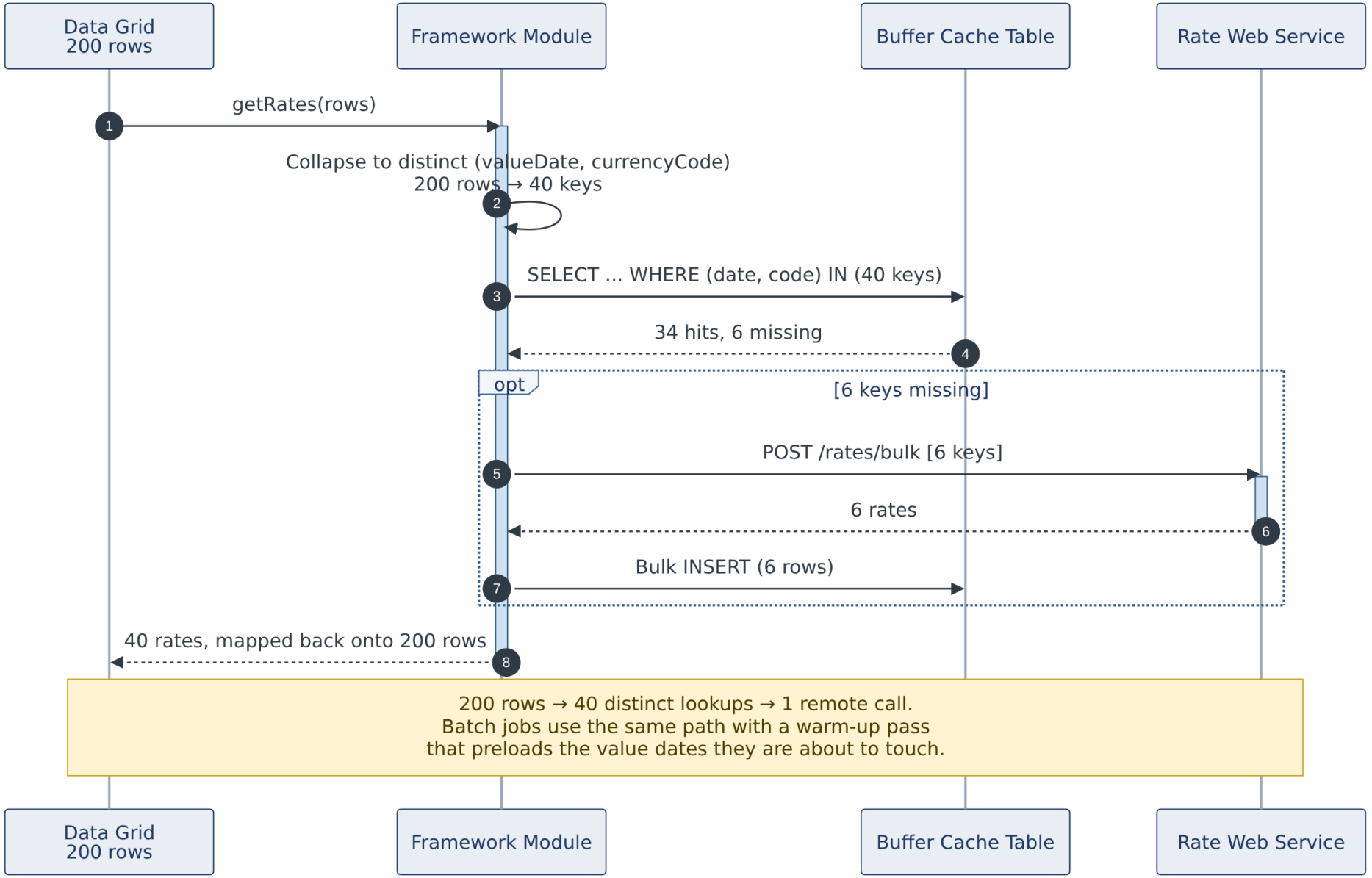
70 ekiple yürütülen bir değişim yönetimi egzersizi

Her uygulama

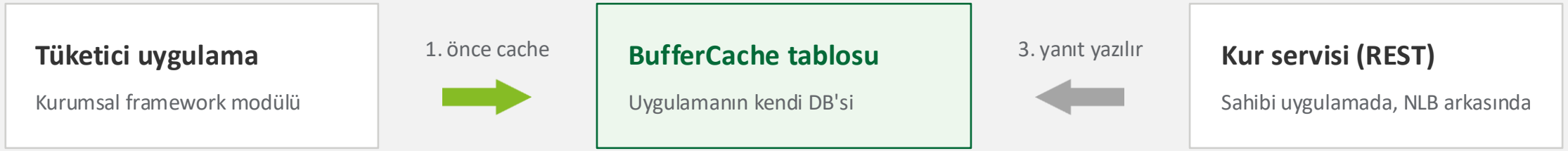
DB bağımlılığı yüzünden taşınamıyor, bölünemiyor, konteynere alınamıyor



Referans veri erişimini bir veritabanı meselesi olmaktan çıkarıp birinci sınıf bir uygulama meselesi hâline getirdik.


Data Grid
200 rows
Framework Module
Buffer Cache Table
Rate Web Service
getRates(rows)
1
Collapse to distinct (valueDate, currencyCode)
200 rows → 40 keys
2
SELECT ... WHERE (date, code) IN (40 keys)
3
34 hits, 6 missing
4
opt
[6 keys missing]
POST /rates/bulk [6 keys]
5
6 rates
6
Bulk INSERT (6 rows)
7
40 rates, mapped back onto 200 rows
8
200 rows → 40 distinct lookups → 1 remote call.
Batch jobs use the same path with a warm-up pass
that preloads the value dates they are about to touch.
Data Grid
200 rows
Framework Module
Buffer Cache Table
Rate Web Service

Yeni Model: Servis + Buffer Cache



2. miss: toplu tarih sorgusu servise gider · yapılandırılabilir retry / circuit-breaker

Kopya taşıyan veritabanları ve senkronizasyon eforu ortadan kalkıyor; şema, sahibi uygulamanın iç detayı oluyor.

Sorgu Yolu: Önce Cache, Gerekirse Servis

1. Tarih + kur kodu ile BufferCache'e bak
2. Yoksa servisi çağır; birden fazla tarih tek çağrıda
3. Yanıtı tabloya yaz, çağırana dön
4. ! Servis erişilemezse: retry, sonra katman hatası

BASİTLEŞTİRİLMİŞ AKIŞ

```
rates = cache.find(dates, code)
missing = dates − rates.dates
if (missing) {
fetched = service.query(missing, code)
cache.save(fetched)
rates += fetched
}
return rates
```

KARAR 1 / 4

Neden Paylaşımlı Şema Yerine Servis?

Sahiplik

Veri sahibi uygulama sözleşmeyi yönetir.

Şema İç Detay

Tüketiciyi bozmadan evrilebilir.

Taşınabilirlik

Uygulama DB bağımlılığı olmadan taşınır, konteynere alınır.

Okuma hızı korunuyor; kırılan şey bağımlılık.

KARAR 2 / 4

Neden DB Tabanlı Cache?

Join

View ve raporlar BufferCache'e join yapabilir.

Kalıcılık

Yeniden başlatmada kaybolmaz; joblar aynı veriyi görür.

Ek Altyapı Yok

Her uygulamanın zaten bir veritabanı var.

In-memory veya dağıtık cache yerine. Saklama ve yenileme davranışı config ile uygulama bazında ayarlanır.

KARAR 3 / 4

Neden REST, Neden gRPC Değil?

Payload Küçük

Birkaç sayı; çağrıların çoğu cache'ten karşılanıyor.

Altyapı Uyumu

NLB, izleme ve güvenlik araçları HTTP ile doğrudan çalışıyor.

Tek Client

Client framework içinde; protokol tüketiciye şeffaf.

Protokol framework'ün arkasında: gerekirse değiştiririz, bugün ihtiyaç yok.

KARAR 4 / 4

Neden MQ Değil?

Desen

İhtiyaç senkron sorgu; geçmiş tarihin kuru event ile gelmez.

Hacim

Günde 2 güncelleme; push altyapısını gerekçelendirmiyor.

Yetkinlik

MQ teknolojileri var, ama 70 ekipte yaygın değil.

Kritik veriyi, ekiplerin olgun olmadığı bir teknolojiye bağlamak operasyonel risk.

200 Satır, 40 Farklı Değer Tarihi

200
satır

→

40
distinct tarih

→

1
toplu çağrı

ADIM 1
Distinct tarih setini çıkar

ADIM 2
Cache'te olmayanları bul

ADIM 3
Tek toplu REST çağrısıyla doldur

ADIM 4
Hesaplamayı yerel veriyle yap

Yavaş Bir Servis, Yavaş Bir Bankaya Dönüşmemeli

1 · Retry

Yapılandırılabilir deneme sayısı

2 · Circuit-breaker

Sağlıksız servise yük bindirmez

3 · Katman hatası

Çağırana açık hata döner; tahminle devam edilmez



Finansal veride yanlış sonuç, hatadan daha kötüdür.

Cache'e Yalnızca Onaylı Veri Girer



Taslak Sızmaz

Onay bekleyen kur hiçbir tüketiciye ulaşmaz.

Onay Sonrası Düzeltme

[Düzeltme senaryosu: nasıl işlediğini buraya yaz]

Göç: Önce En Yüksek Kazanç, En Düşük Risk

TAMAMLANDI

İlk Dalga

Veri kullanımı yüksek, değişimi kolay uygulamalar

DEVAM EDİYOR

Sonraki Dalgalar

Kalan uygulamalar, sahipleriyle koordineli

HEDEF

Kapanış

Joblar ve paylaşımlı DB'ler kaldırılır

Geçişte iki kaynak yok: aynı onaylı verinin iki dağıtım kanalı var.

Neyi Farklı Yapardık?

1

Hibrit Invalidation

Sorgu serviste kalır; düzeltme bildirimi event ile gelir.

2

Ölçüm Baştan

Hit oranı, miss gecikmesi ve servis hata oranı ilk günden.

3

Önce Envanter

Paylaşımlı DB'ye erişen SP, view, rapor ve joblar göçten önce.

Taşınabilir Desen

Kurlar, parametreler, kod listeleri, takvimler: başkasının veritabanından okunan her referans veri

1

Sahibi Belirle

Veriye sahibi uygulamada bir servis

2

Erişimi Standartlaştır

Framework modülü, tek client

3

Yerelde Tut

Tüketicide buffer cache tablosu

4

Dürüst Hata Ver

Retry, circuit-breaker, fail-fast

Teşekkürler

Sorular & Tartışma

“Ne kadar bilirsen bil, söylediklerin karşındakinin anlayabildiği kadardır.”
— Mevlana

in linkedin.com/in/ibrahimatay 𝕏 x.com/ibrahim_atay ⬢ github.com/ibrahimatay

